Do LLM Judges Penalise the Script? A Script-Controlled Audit of LLM-as-Judge Scoring for Hindi in Devanagari versus Romanized Orthographies

Mohammed Abraar
Vizuara Research
abraar@vizz.vizuara.ai

Abstract

Large language models now grade other language models, feeding leaderboards, reward models, and quality gates that all assume the writing system of an answer does not move its score. We test that assumption for Hindi, written natively in Devanagari and, by hundreds of millions online, in the Latin alphabet. We hold content fixed and vary only the script: 150 instruction-response pairs at three quality tiers, each in Devanagari and three romanizations (scholarly IAST, diacritic-stripped ASCII, user-style Hinglish), scored blind by seven judges from four families. We pre-specified the prediction that romanized text would score lower; the data reversed it. First, Gemini 3.6 Flash scores identical romanized content 2.2 to 3.4 points higher than Devanagari on a 100-point scale (all $p \leq 0.003$), concentrating at 6.5 to 7.8 points on medium-quality responses, where a judge has the most discretion; Gemini 3.1 Pro shows the same sign at one-third to one-half the size. Second, Qwen2.5-7B inflates scholarly IAST by 7.6 and ASCII by 6.0 points, reaching $+\mathbf{17.8}$ on low-quality items yet flat on Hinglish. Third, Claude judges move opposite, the direction we pre-registered: one item per call, Sonnet 5 penalises ASCII by 12.3 points, replicating in direction through a matched temperature-0 gateway (-3.6, high tier -9.2) and on a non-Claude-authored item set (high tier -16.7), with Opus 5 smaller, Fable 5 near-invariant, GPT-5.6 near-robust, a suggestive capability ordering. Fourth, protocol decides what you see: with batch size the only varied factor, batching erases Sonnet's -12.3 entirely ($\rightarrow +1.1$), and in pairwise trials all seven judges prefer the Devanagari twin of identical content, six of them with zero ASCII wins in 300 trials each ($p < 10^{-85}$), reversing even the judges whose pointwise scores inflate ASCII. Fifth, prompting does not fix it: mitigation is at best partial on Gemini, flips sign under a rubric variant, and backfires on Qwen (up to $+11.8$ from a $+7.6$ baseline). A per-dimension decomposition shows Sonnet's penalty is 68 percent a clarity judgment but leaks significantly into correctness, which identical content cannot differ on. Mechanism probes rule out token length ($|r| \leq 0.18$ with score shift) while rationales name the orthography on 41 to 57 percent of romanized items. Script bias in LLM judges is real, sign-opposite across families, protocol-sensitive in magnitude and sign, and not reliably promptable away.

1 Introduction

Evaluation of language models has largely been handed to other language models. An LLM judge reads a question and an answer and returns a score (Zheng et al., 2023) that then ranks systems on leaderboards, trains the next model as a reward signal, and gates what production systems ship. The reliability literature has catalogued many failure modes of these judges, including position bias (Wang et al., 2024), verbosity and self-enhancement bias (Zheng et al., 2023), self-preference driven by self-recognition (Panickssery et al., 2024),

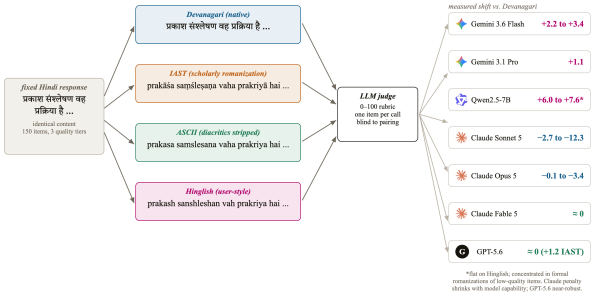

Figure 1: Study design. One Hindi response, four orthographies, seven blind judges. Any score gap is script bias by construction; measured outcomes previewed at right (§4).

and taxonomies now spanning a dozen or more bias types (Ye et al., 2025; Zhou et al., 2026b). Every catalogue so far shares one silent assumption: that the writing system of the judged text does not move the score.

That assumption matters most for languages with two written lives, and Hindi is the clearest case: its official script is Devanagari, but a very large share of Hindi on phones and social media is typed in the Latin alphabet, romanized Hindi or Hinglish (Sheth et al., 2025). The two forms carry identical content; a judge that scores them differently is measuring orthography, not quality.

This paper asks one narrow question with a clean design: hold the language and content of an answer fixed, change only its script, and ask whether an LLM judge's score moves. Because the content on both paths is identical (Figure 1), nothing else can explain a gap. Prior multilingual judge studies compared different languages, entangling language with script (Hada et al., 2024b; Fu and Liu, 2025; Zhou et al., 2026a); we untangle them.

We recorded the predicted direction in the project log before collecting data: by analogy to the task-performance literature, where romanized input degrades accuracy by up to 24 points (Khullar et al., 2025) and inflates perplexity over 300-fold (Rajapakse and Weerasinghe, 2026), we expected romanized text to score lower. The data reversed the prediction: romanized Hindi scores higher on Gemini and Qwen judges, the inflation concentrates where a judge has discretion, and it persists or worsens when the judge is told to ignore the script. Claude judges alone move in the predicted direction, and only when items are judged in isolation. We report the reversal as found.

Our contributions:

- Script-controlled judge audit (§3): the first audit isolating script as a treatment: 150 fixed Hindi items, four orthographies, seven judges, four families, 19,806 scripted judgments (4,200 core, 600 matched-gateway core, 2,400 protocol arms, 7,200 mitigation-battery, 1,200 sub-dimension, 2,100 pairwise, 600 cross-script, 600 test-retest, 306 authorship-replication, 300 anchored, 300 ranking-demo).
- A new, sign-opposite judge-bias axis (§4): up to +7.8 points of romanization inflation under Gemini 3.6 Flash and +17.8 under Qwen2.5-7B, against a penalty of up to −12.3 under Claude Sonnet 5 that shrinks with capability; orthography appears in no existing judge-bias taxonomy (Ye et al., 2025; Zhou et al., 2026b).
- A protocol effect that masks, and can even reverse, the bias (§4.3): a single-variable control shows batching alone erases Sonnet's −12.3 ASCII penalty, and pairwise judging flips Flash's pointwise inflation into near-total Devanagari preference (288/300 trials, zero ASCII wins), a warning for judge-bias audits and arena-style leaderboards alike.
- A six-arm mitigation battery that fails in three different ways (§4.4): prompting reduces the bias at best partially, flips its sign at worst, and amplifies it on the open-weights judge; the practitioner remedy is judge selection and script-stratified reporting, not prompting.

- Mechanism probes (§4.5): token length is ruled out as the driver, and the judges' rationales show the script is perceived while the miscalibration happens anyway.

2 Related Work

LLM-as-judge and its biases. Zheng et al. (2023) established the paradigm and catalogued position, verbosity, and self-enhancement biases; Wang et al. (2024) flipped pairwise judgments by candidate order alone; Panickssery et al. (2024) traced self-preference to self-recognition; CALM (Ye et al., 2025) and JudgeBiasBench (Zhou et al., 2026b) quantify a dozen-plus bias types. Orthography is absent from all of these.

Multilingual judging. Hada et al. (2024b) calibrated GPT-4 judging against native speakers across eight languages, finding systematic overscoring, worst for low-resource languages. METAL (Hada et al., 2024a) and PARIKSHA (Watts et al., 2024) measured judge-human agreement across languages; MM-Eval (Son et al., 2024) and Fu and Liu (2025) stress-tested judges across up to 122 languages; BabelJudge (KC, 2026) measures Hindi reliability in native script only (see also Zubiaga et al., 2026). Closest to us, Zhou et al. (2026a) audit 23 languages, find lower-resource ones scored more generously, and note a Latin-script advantage observationally. In every one of these, script co-varies with language; none isolates it.

Script and romanization in LLMs. RomanSetu (Husain et al., 2024) showed romanization can serve as an efficient adaptation interface for Indic languages, building on a longer transliteration-adaptation history (Purkayastha et al., 2023; Xhelili et al., 2024; Madhani et al., 2023). On deployment, Script Gap (Khullar et al., 2025) measured task-accuracy drops up to 24 points on romanized inputs across five Indian languages, Rajapakse and Weerasinghe (2026) measured over 300-fold perplexity degradation on romanized Sinhala, and Indi-RomCoM (Das et al., 2026) benchmarked romanized code-mixed degradation (resources: Sheth et al., 2025; Yang and Chai, 2025). Mechanistically, RomanLens (Saji et al., 2025) finds latent romanization inside multilingual models, and Karne (2026) shows concept representations are not script-invariant. All of this measures the model as a task performer or probes its internals; none measures it as a judge.

Tokenizer inequity. Tokenizers price languages unequally (Petrov et al., 2023; Ahia et al., 2023; Ali et al., 2024; Liang et al., 2023); Indic byte-fallback fragmentation has been quantified as an eight-fold token tax (Srivastava, 2026), with morphology-aware remedies proposed (Limisiewicz et al., 2024; Brahma et al., 2025). This literature stops at token counts and task accuracy; we test, and reject, its natural prediction for judge scores.

Indic evaluation. Indic benchmarks evaluate in the conventional native script only (Ahuja et al., 2023; Doddapaneni et al., 2023; Singh et al., 2024; Verma et al., 2025); Airavata (Gala et al., 2024) exemplifies current practice, its Hindi generations GPT-4-judged in Devanagari with the judge unvalidated. Our result says that practice is sensitive to an unexamined variable.

3 Method

Dataset. We construct 150 Hindi instruction and response pairs (authored by a Claude-family model under human-specified tier definitions; §5 discusses the resulting self-preference exposure) across ten everyday domains, at three intended quality tiers of 50 items each: high (correct, complete, 80 to 140 words), medium (correct but visibly incomplete, 40 to 80 words), and low (curt, vague, or shallow, 15 to 40 words, a few with deliberate minor factual errors). Responses were authored in Devanagari and frozen before any judging. Tier intent was strongly realised: Devanagari means of 99.1, 51.4, and 15.2 under Gemini 3.6 Flash.

Conditions. Each frozen response is rendered in four orthographies. deva is the original Devanagari. iast is scholarly romanization with diacritics, produced deterministically with the indic-transliteration library (Madhani et al., 2023). ascii strips the diacritics from IAST, a proxy for plain-keyboard romanization; the stripping is lossy (long/short vowel and retroflex/dental distinctions merge), which is precisely what plain-keyboard typing loses, so

Table 1: Mean within-item score shift (romanized minus Devanagari), bootstrap 95% CIs, n=150. Bold: permutation $p < 0.01$. All judges scored one item per call (§4.3).

Judge	IAST	ASCII	Hinglish
Gemini 3.6 Flash	**+3.43** [+2.4, +4.6]	**+2.17** [+0.8, +3.7]	**+3.13** [+2.2, +4.1]
Gemini 3.1 Pro	**+1.17** [+0.6, +1.8]	**+1.10** [+0.4, +1.8]	**+1.13** [+0.5, +1.8]
Qwen2.5-7B	**+7.63** [+4.9, +10.4]	**+6.03** [+3.3, +8.7]	+0.07 [−1.8, +1.9]
Claude Sonnet 5	**−2.70** [−4.7, −0.8]	**−12.29** [−15.6, −9.2]	**−2.71** [−4.0, −1.4]
Claude Opus 5	−0.13 [−0.8, +0.6]	**−3.43** [−4.4, −2.5]	**−1.18** [−1.9, −0.5]
Claude Fable 5	**+1.43** [+1.0, +1.9]	−0.30 [−1.1, +0.4]	+0.13 [−0.3, +0.6]
GPT-5.6	+1.21 [+0.2, +2.2]	+0.51 [−0.6, +1.7]	+0.62 [−0.2, +1.5]

the ascii cell tests a degraded-but-ecological form while iast tests a lossless one. hinglish is a natural user-style respelling (for example kaise hain aap), produced under a strict no-paraphrase rule: every word, the word order, and all punctuation preserved. In every condition the instruction is rendered in the same orthography as the response (the ascii cell uses the IAST instruction), so a romanized response never answers a Devanagari question; §4.6 also measures the deliberately mismatched cells. A native-speaker audit of a stratified 20-item sample of the hinglish condition confirmed 20/20 preserve content exactly; the audited sample ships with the released data.

Judges and protocol. Seven judges from four families score every item in every condition on a 0 to 100 rubric (correctness, completeness, helpfulness, clarity). Gemini 3.6 Flash and Gemini 3.1 Pro Preview are called through the API at temperature 0, one item per call, in randomized order, so the judge never sees two versions of the same item in one context. Claude Sonnet 5, Opus 5, and Fable 5 judge through blind agent sessions arranged in a Latin square: each session sees each item exactly once, in exactly one condition, with conditions rotated across four sessions. Qwen2.5-7B-Instruct runs open-weights under vLLM on one A10G GPU at temperature 0, one item per call, with top-20 logprobs recorded at the score position. GPT-5.6 is called one item per call at temperature 0 through a unified OpenAI-compatible gateway. No judge is told the study concerns scripts. The judging prompt asks for a score and a one-sentence reason; the reasons become data for the perception analysis (§4.5).

Follow-up batteries. Eight targeted batteries close the main alternative explanations (§4.6): a protocol control varying only batch size on a fixed interface; a per-dimension rubric decomposition on the main arms; an order-counterbalanced pairwise preference test; a cross-script control measuring the deliberately mismatched instruction/response cells; test-retest baselines for a deterministic and a stochastic judge; an anchored-pointwise variant with an in-context reference; and Benjamini-Hochberg FDR correction plus a logit-scale tier reanalysis. Verbatim prompts and full tables are in the appendix.

Analysis. The unit of analysis is the within-item paired difference, romanized minus Devanagari, per judge and condition: mean shifts with bootstrap 95 percent CIs (10,000 resamples), two-sided sign-flip permutation tests (20,000 permutations), and paired d_z. Every number is produced by scripted analyses reading the raw score files; the outputs ship with the paper.

Pre-registration. Before data collection we recorded the predicted direction: romanized scores lower than Devanagari. The observed direction is mostly the opposite; we report all pre-specified analyses regardless.

4 Results

4.1 Three families, three bias profiles

Table 1 and Figure 2 are the study in one view. Gemini 3.6 Flash scores the same content higher when romanized: +3.43 for IAST ($p = 5 \times 10^{-5}$, $d_z = 0.49$), +2.17 for ASCII ($p = 0.003$), +3.13 for Hinglish ($p = 5 \times 10^{-5}$, $d_z = 0.52$); Gemini 3.1 Pro shows the same sign at one-third to one-half the size (all $p \leq 0.003$).

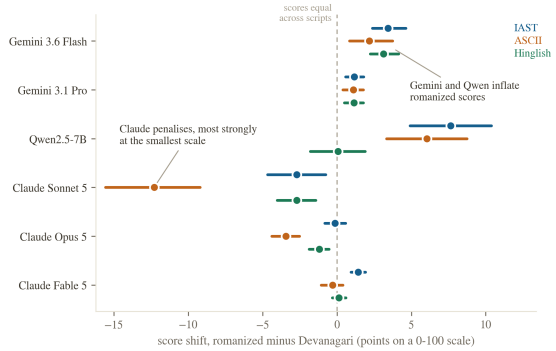

Figure 2: Score shift by judge and romanization, 95% bootstrap CIs, $n = 150$. Right of the dashed zero line means romanized scored higher.

The open-weights judge changes shape. Qwen2.5-7B inflates formal romanizations dramatically, +7.63 IAST and +6.03 ASCII, but is flat on natural Hinglish. Its inflation lives elsewhere too: low-quality items shift +17.8 (IAST) and +17.4 (ASCII), the small judge failing to recognise bad content dressed in diacritics, while Hinglish, plentiful in web data (Sheth et al., 2025), is judged as soberly as Devanagari.

The Claude family moves in the opposite direction, the one we pre-registered. Judged one item per call, Sonnet 5 penalises every romanization: -2.70 for IAST ($p = 0.007$), -12.29 for ASCII ($p = 5 \times 10^{-5}$, $d_z = -0.62$), and -2.71 for Hinglish ($p = 10^{-4}$). Opus 5 shows the same sign at a fraction of the size (-3.43 on ASCII, -1.18 on Hinglish), and Fable 5 is near-invariant, with one small inflation on IAST (+1.43). Within one family, then, the script penalty shrinks monotonically with model capability, and the pattern extends across families: GPT-5.6 is likewise near-robust (largest shift +1.21 on IAST, $p = 0.02$; ASCII and Hinglish indistinguishable from zero), though it still inflates low-tier romanized items by +3.1 to +3.8. Sonnet's rationales say why out loud: they call romanized responses "awkward IAST-transliterated Hindi rather than natural Devanagari," treating the orthography itself as a quality defect. Because our rubric asks for a clarity sub-score, and stripped ASCII is plausibly less clear to a competent Hindi reader, part of the Sonnet penalty could be a legitimate clarity judgment; a per-dimension decomposition (§4.6) confirms clarity carries 68 percent of it but shows a significant residue on correctness and helpfulness, dimensions identical content cannot differ on.

Three conclusions follow. Script bias in LLM judges exists, in both directions: the same romanized answer is overpaid by one family and taxed by another. It is a property of the judge, not the task: same items, same protocol, three behaviours across four families, with a capability gradient. And it is not monotone in naturalness: Gemini inflates every romanization, Qwen inflates the formal romanizations it saw least in training, and Claude taxes hardest the stripped-ASCII form that most visibly degrades the text.

4.2 The bias lives where the judge has discretion

Figure 3 splits the shift by quality tier. Under Flash, high-quality items shift at most 1.3 points against a ceiling near 99, low-quality at most 2.7 against a floor near 15, while medium-quality items, near 51 in Devanagari, inflate by +7.8 (IAST), +7.4 (ASCII), and +6.5 (Hinglish), all $p \leq 1.5 \times 10^{-4}$. The same shape holds for Pro at smaller magnitude. This is the pattern one expects if romanization blurs the judge's view of quality: content whose flaws are obvious stays low, content whose merits are obvious stays high, and ambiguous content drifts toward a generous middle. It is also the pattern a purely mechanical ceiling-and-floor compression would produce with no discretion story at all; a logit-scale reanalysis (§4.6), which removes exactly that compression, still finds the medium tier's shift largest, so the discretion reading survives the transform. Appendix Figure 6 shows one such drift verbatim: one refrigerator explanation, four orthographies, a thirty-point spread under one judge.

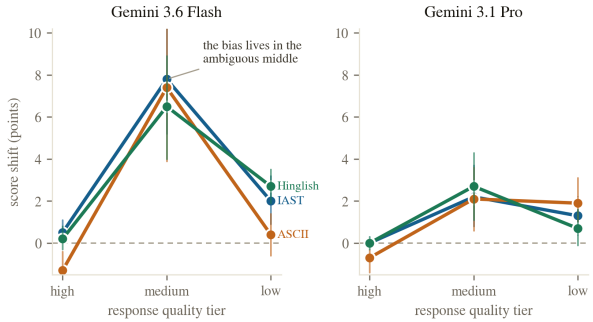

Figure 3: Shift by quality tier, Gemini judges, 95% bootstrap intervals, $n = 50$ per tier. High/low tiers sit near the score ceiling/floor.

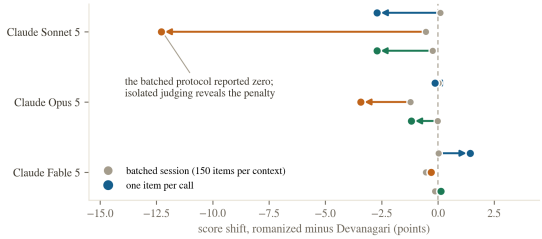

Figure 4: Two protocols, two answers. Batched sessions (grey) versus one item per call (colored) for the same three Claude judges; batching reports near-zero, isolation reveals penalties up to −12.3.

4.3 The measurement protocol can hide the bias

Our first Claude measurements used batched agent sessions, 150 items per context with conditions rotated in a Latin square; all three Claude judges appeared script-invariant, shifting at most 0.54 points in eight of nine comparisons. Rerunning the identical items one per call flipped the answer (Figure 4): the invariance was a batch artifact. Because those two arms also differed in interface and decoding temperature, we ran a single-variable control: the same Sonnet 5, the same headless interface, the same per-item rubric and sampling, with items grouped 25 per call. Batch size alone reproduces the masking: the −12.29 ASCII penalty becomes +1.12, and the high-tier penalty collapses from −24.3 to −3.6 (full table in the appendix). A judge that has just scored dozens of items appears to anchor its scale on content quality and stops reacting to surface form; in isolation, the orthography moves the score. Batched audits, common because they are cheap, can therefore mask a twelve-point bias production one-per-call judging would feel in full.

4.4 Prompting does not reliably fix it

The remedy every practitioner would try first is a line in the judge prompt. We tested six: a plain instruction, an explicit warning naming both scripts, a fairness framing, a transliterate-first instruction, a script-blind persona, and a decomposed rubric forcing four sub-scores summing to 100; each ran over the full four-condition protocol on two judge families (Figure 5).

On Gemini 3.6 Flash, no arm closes the gap. The best, transliterate-first, cuts medium-tier inflation from +7.8 to +4.4; the persona arm does nothing (+7.7); and the decomposed rubric overshoots, flipping ASCII from +7.4 to −2.8, trading inflation for penalty rather than removing bias. On Qwen2.5-7B the battery backfires uniformly: every instruction increases IAST inflation above the +7.6 no-instruction baseline, up to +11.8 under the fairness framing, and the script-blind persona even induces a +4.6 Hinglish inflation where the baseline judge was flat. Mentioning scripts in the prompt appears to direct the smaller judge's attention to the very feature it should ignore. Prompt-level mitigation is therefore

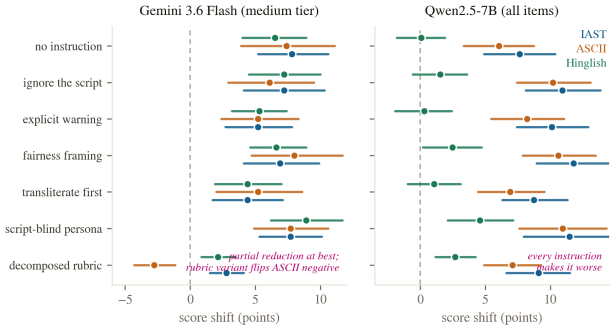

Figure 5: Six mitigation arms, two families, no reliable fix. Left: Gemini 3.6 Flash, medium tier, $n = 50$. Right: Qwen2.5-7B, all items, $n = 150$: every arm increases inflation.

unreliable in the strongest sense: its effect ranges from partial reduction through sign reversal to amplification, consistent with debiasing results showing prompt-level fixes underperform training-level ones (Zhou et al., 2026b). The practitioner remedy remains judge selection and script-stratified reporting, not prompting. The mitigation battery tests pointwise absolute scoring; whether these instructions transfer to pairwise preference judging is untested, and §4.6 shows the pairwise protocol itself behaves very differently.

4.5 Mechanism: not token length; the script is perceived

Two probes narrow the explanation space. First, tokenization: the tokenizer-fairness literature predicts length disparities drive downstream behaviour (Petrov et al., 2023; Ahia et al., 2023; Srivastava, 2026). Measured with the Gemini tokenizer, romanized Hindi is the expensive form, costing 1.98 (IAST), 1.47 (ASCII), and 1.33 (Hinglish) times the Devanagari tokens (Appendix Figure 7, left), itself a finding: it reverses the romanization-efficiency premise of 2 to 4-fold token savings (Husain et al., 2024); modern tokenizers have closed the gap (Brahma et al., 2025). But token inflation does not explain the score inflation: the per-item correlation between token ratio and score shift is $r = +0.11$ (IAST), $+0.05$ (ASCII), and -0.08 (Hinglish), and no tier-restricted correlation exceeds $|r| = 0.18$. Longer input is not what earns the higher score. The top-20 logprobs recorded at the Qwen score position offer a partial distributional view: because they were captured at the first generated token only, they describe the leading digit of the score, not the full 0-100 value. On that limited view, romanized conditions shift the leading-token expectation upward (IAST 6.33 versus Devanagari 5.70) with essentially unchanged within-call spread, consistent with a mode move rather than a widening; a full-sequence distributional analysis remains future work.

Second, perception. The judge's rationales show it sees what it is scoring (Appendix Figure 7, right): under Flash, 47 percent of IAST, 57 percent of ASCII, and 41 percent of Hinglish rationales explicitly mention the script, against 1 percent for Devanagari; some name the orthography as a deficiency while scoring the item above its Devanagari twin. The bias operates despite awareness, echoing behaviourally the finding that internal representations are not script-invariant (Karne, 2026; Saji et al., 2025).

4.6 Follow-up batteries: closing the confounds

Eight targeted batteries (4,500 judgments beyond the original 12,600: 600 GPT core and 600 batched control, already included in the core and protocol categories of §3's total, plus 1,200 sub-dimension, 600 pairwise, 600 cross-script, 600 test-retest, and 300 anchored; tables in the appendix) address the main alternative explanations.

Noise floor. Thirty stratified items, deva and ascii, five identical calls each: mean within-item standard deviation is 1.66 points for Flash at temperature 0 and 2.00 for Sonnet under the headless interface's default sampling. Sonnet's -12.3, Qwen's $+6.0$ to $+7.6$, and the

medium-tier inflations sit five to seven of these standard deviations out; shifts near ±1 do not.

Is the penalty just clarity? Under the decomposed rubric, Sonnet's ASCII shift totals −7.14: clarity carries 68 percent, but correctness drops −1.57 ($p = 5 \times 10^{-5}$) and helpfulness −0.54 ($p = 0.001$), dimensions identical content cannot differ on. Flash's shift is pure clarity (−2.29). The non-clarity residue is bias by construction.

Is it the interface? Rerunning the full Sonnet core arm through the same OpenAI-compatible gateway as GPT-5.6, at temperature 0, one item per call, the ASCII penalty replicates in direction and tier structure at reduced magnitude (−3.64, $p = 5 \times 10^{-5}$; high tier −9.24), while the smaller IAST and Hinglish penalties do not (+0.49, $p = 0.40$; +1.89). The sign reversal against Gemini is interface-robust; the headline magnitude, like everything else here, is protocol-sensitive.

Is it self-preference? The 150 items were Claude-authored (§5), so we authored a 51-item replication set with Gemini 3.1 Pro and reran both key judges. Sonnet's high-tier ASCII penalty replicates on items no Claude model wrote (−16.71, CI [−21.0, −12.5], $p = 10^{-4}$), and Flash's inflation replicates throughout (+4.61 IAST, $p = 5 \times 10^{-5}$). Self-recognition cannot explain a penalty on another family's text.

Is it question-answer script mismatch? No: an ASCII response answering a Devanagari instruction scores 13.5 (Flash) and 16.1 (Sonnet) points below the same response with a script-matched instruction (both $p = 5 \times 10^{-5}$), so mismatch is a separate, larger penalty that every headline number excludes by construction (§3).

Does pairwise judging agree? No, starkly, and unanimously: shown the deva and ascii twins in both orders, every one of the seven judges prefers Devanagari (Flash 288/300, Sonnet 293/300, Pro 299/300, Opus and Fable 300/300, GPT-5.6 283/300, Qwen 252/300; all sign-test $p < 10^{-63}$). Qwen's graded pattern (8 ASCII wins, 40 ties) shows the format can produce ties and ASCII wins, ruling out a degenerate-prompt artifact; the capable judges simply never choose ASCII. Flash inflates ASCII by +2.2 pointwise yet prefers Devanagari 96 percent of the time pairwise: the two standard protocols disagree not in degree but in sign, and preference-trained reward models sit on the pairwise side. An anchored-pointwise variant (the frozen Devanagari original as an in-context reference) points the same way: the ASCII inflation vanishes (−0.97, $p = 0.65$) and reverses to −10.4 on high-quality items, suggesting the pairwise verdict is the anchored assessment and unanchored pointwise is the outlier; the deva cell's response equals its reference, so this is directional evidence only.

Does a leaderboard actually move? Three real systems (Qwen2.5-7B, Flash, Pro) answered 50 of our instructions in Hindi; Sonnet 5, a non-contestant, scored all 300 outputs in both scripts. The three-way ranking survives, but the script moves systems differentially: Flash −6.1, Pro −5.2, Qwen +2.8 under romanization. These per-system shifts are the same order as the real Flash-Pro gap (≈ 5 points), so systems separated by less than a judge's script shift can swap places; our pair survived by about a point.

Multiple comparisons and scale artifacts. Benjamini-Hochberg correction over all 198 reported p-values leaves 108 significant at $q = 0.05$ (effective threshold $p \leq 0.0249$), including every headline effect. A logit-transform reanalysis of the tier pattern, which removes ceiling-and-floor compression, still places the largest shift at the medium tier for all three romanizations, so the discretion account survives (§4).

5 Limitations, narrated

We narrate the study's weaknesses as part of the story; several were speed-and-budget design choices a reader should weigh.

There is no human reference score anywhere in this study. Every headline number is a shift relative to Devanagari, an anchor choice rather than validated ground truth: the data equally support judges under-scoring Devanagari, and words like "inflates" and "penalises" assert a direction of error the design cannot establish; read them as shorthand for "shifts relative to Devanagari" pending a human-anchored replication. The decomposition (§4.6)

bounds the related clarity concern but cannot say whether the clarity component matches what a human reader experiences.

The dataset is model-authored: all 150 items and their tiers were generated by a language model, then frozen; model-authored Hindi may differ from human Hindi in ways that interact with romanization. The hinglish condition is a model respelling under a strict no-paraphrase rule; a native-speaker audit of a stratified 20-item sample confirmed 20/20 preserve content, word order, and punctuation exactly, though the audit covers only hinglish, not IAST or ASCII, and does not certify the content as representative of human Hindi.

The Claude judges were measured under two protocols: batched agent sessions and headless one-item-per-call calls to the pinned model ids, the latter using default sampling where the API arms pin temperature 0. The batching control attributes the masking to batch size alone, and the matched-gateway arm (§4.6) now shows the Gemini-versus-Sonnet ASCII sign reversal survives on one interface at temperature 0; but the magnitude gap between the two Sonnet arms (-12.3 versus -3.6) is itself unexplained variance a fuller interface study should decompose. The 150 items were authored by a Claude-family model, so a self-recognition account (Panickssery et al., 2024) initially predicted the Claude judges' preference for the authored surface form. The Gemini-authored replication set (§4.6) now excludes this for the headline effect: the high-tier ASCII penalty survives on another family's text. Smaller Claude-direction effects retain the exposure, and the full 150-item set remains single-authored.

The core arms ran one seed per item per condition; the test-retest batteries (§4.6) now provide noise yardsticks for both a deterministic and a stochastic judge, and the headline effects clear them several-fold, but per-item shifts near a point remain indistinguishable from sampling noise. The ranking demo (§4.6) shows differential per-system script shifts of leaderboard-relevant size, but an observed flip between closely matched real systems remains undemonstrated. FDR correction is now reported (§4.6); individual small cells that do not survive it (90 of 198 tests) should be read as unconfirmed.

Scale and scope are modest: one language, 150 items, seven judges, six prompt-level mitigation arms on two families, no training-level intervention. The quality tiers are coarse; the logit reanalysis (§4.6) defends the discretion account against scale compression, but a continuous-quality design would test it more sharply. The capability ordering rests on four checkpoints across two families with no independent capability metric: suggestive, not a scaling law. Extending to other digraphic settings (Karne, 2026; Rajapakse and Weerasinghe, 2026) is the natural next step.

Finally, the pre-registration was directional and informal (a project-log entry, not a registry filing); we state it because it disciplined our reporting of a reversed prediction.

6 Conclusion

We held Hindi content fixed, changed only its writing system, and asked seven LLM judges to grade it. The writing system moved the grades, in both directions: Gemini judges inflate romanized Hindi by 2 to 3 points overall and 7 to 8 on ambiguous-quality content; a 7B open-weights judge inflates formal romanizations by up to 18 points on low-quality content; Claude judges tax romanization instead, up to 12.3 points at the smallest scale in a suggestive capability ordering, while GPT-5.6 is near-robust; batch size alone erases the largest penalty; the judge that inflates ASCII pointwise prefers Devanagari in 96 percent of pairwise trials; the penalty is majority clarity but leaks into correctness; six mitigations range from partial to counterproductive; token length explains none of it; and the rationales prove the script is seen. The bias axis is real, judge-dependent in size and sign, protocol-sensitive in magnitude and sign, and it belongs in every judge-bias taxonomy (Ye et al., 2025; Zhou et al., 2026b) and multilingual evaluation checklist (Hada et al., 2024b; Son et al., 2024), though establishing it as judge error rather than non-invariance still requires a human anchor (§5). Until judges are audited for script invariance, Hindi evaluations should report native-script and romanized slices separately, and should say which judge produced the numbers.

Reproducibility Statement

All datasets, per-call score files, the scripted analysis, figure build scripts, and spend logs accompany this submission and will be released publicly upon acceptance. Every number in this paper traces to a single analysis script reading the raw judgment files, described in §3. Total marginal compute cost of the study was under 6 US dollars of metered API spend plus roughly one GPU-hour on a single accelerator for the open-weights judge; Claude judging ran through a separate subscription-billed interface not captured in that figure.

LLM Usage Statement

Large language models were used as the subjects of this study (the seven judges under audit) and as engineering assistants for writing analysis code, drafting figures, and copy-editing this manuscript. All experimental design, statistical choices, interpretation of results, and claims are the authors'; every number reported was checked against the scripted analysis output described in §3 before being written into the text.

References

Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David R. Mortensen, Noah A. Smith, and Yulia Tsvetkov. Do all languages cost the same? Tokenization in the era of commercial language models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023.

Kabir Ahuja, Harshita Diddee, Rishav Hada, Millicent Ochieng, Krithika Ramesh, Prachi Jain, Akshay Nambi, Tanuja Ganu, Sameer Segal, Maxamed Axmed, Kalika Bali, and Sunayana Sitaram. MEGA: Multilingual evaluation of generative AI. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023.

Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leveling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Schulze Buschhoff, Charvi Jain, Alexander Arno Weber, Lena Jurkschat, Hammam Abdelwahab, Chelsea John, Pedro Ortiz Suarez, Malte Ostendorff, Samuel Weinbach, Rafet Sifa, Stefan Kesselheim, and Nicolas Flores-Herr. Tokenizer choice for LLM training: Negligible or crucial? In Findings of the Association for Computational Linguistics: NAACL 2024, 2024.

Maharaj Brahma, N J Karthika, Atul Singh, Devaraj Adiga, Smruti Bhate, Ganesh Ramakrishnan, Rohit Saluja, and Maunendra Sankar Desarkar. MorphTok: Morphologically grounded tokenization for indian languages. In Tokenization Workshop (TokShop) at ICML 2025, 2025.

Avisha Das, Mihir Parmar, Mohana Ramnath, and Pulkit Verma. Indi-RomCoM: Code-mixed benchmark for evaluating LLMs on romanized Indic-English instructions. arXiv preprint arXiv:2606.30790, 2026.

Sumanth Doddapaneni, Rahul Aralikatte, Gowtham Ramesh, Shreya Goyal, Mitesh M. Khapra, Anoop Kunchukuttan, and Pratyush Kumar. Towards leaving no indic language behind: Building monolingual corpora, benchmark and models for indic languages. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023.

Xiyan Fu and Wei Liu. How reliable is multilingual LLM-as-a-Judge? In Findings of the Association for Computational Linguistics: EMNLP 2025, 2025.

Jay Gala, Thanmay Jayakumar, Jaavid Aktar Husain, Aswanth Kumar M, Mohammed Safi Ur Rahman Khan, Diptesh Kanojia, Ratish Puduppully, Mitesh M. Khapra, Raj Dabre, Rudra Murthy, and Anoop Kunchukuttan. Airavata: Introducing Hindi instruction-tuned LLM. arXiv preprint arXiv:2401.15006, 2024.

Rishav Hada, Varun Gumma, Mohamed Ahmed, Kalika Bali, and Sunayana Sitaram. METAL: Towards multilingual meta-evaluation. In Findings of the Association for Computational Linguistics: NAACL 2024, 2024a.

Rishav Hada, Varun Gumma, Adrian de Wynter, Harshita Diddee, Mohamed Ahmed, Monojit Choudhury, Kalika Bali, and Sunayana Sitaram. Are large language model-based evaluators the solution to scaling up multilingual evaluation? In Findings of the Association for Computational Linguistics: EACL 2024, 2024b.

Jaavid Aktar Husain, Raj Dabre, Aswanth Kumar, Jay Gala, Thanmay Jayakumar, Ratish Puduppully, and Anoop Kunchukuttan. RomanSetu: Efficiently unlocking multilingual capabilities of large language models via romanization. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024.

Sripad Karne. One language, two scripts: Probing script-invariance in LLM concept representations. arXiv preprint arXiv:2603.08869, 2026.

Shreyas KC. BabelJudge: Measuring LLM-as-a-Judge reliability across languages and agent trajectories. arXiv preprint arXiv:2606.22329, 2026.

Manurag Khullar, Utkarsh Desai, Poorva Malviya, Aman Dalmia, and Zheyuan Ryan Shi. Script gap: Evaluating LLM triage on indian languages in native vs romanized scripts in a real world setting. arXiv preprint arXiv:2512.10780, 2025.

Davis Liang, Hila Gonen, Yuning Mao, Rui Hou, Naman Goyal, Marjan Ghazvininejad, Luke Zettlemoyer, and Madian Khabsa. XLM-V: Overcoming the vocabulary bottleneck in multilingual masked language models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023.

Tomasz Limisiewicz, Terra Blevins, Hila Gonen, Orevaoghene Ahia, and Luke Zettlemoyer. MYTE: Morphology-driven byte encoding for better and fairer multilingual language modeling. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024.

Yash Madhani, Sushane Parthan, Priyanka Bedekar, Gokul NC, Ruchi Khapra, Anoop Kunchukuttan, Pratyush Kumar, and Mitesh M. Khapra. Aksharantar: Open indic-language transliteration datasets and models for the next billion users. In Findings of the Association for Computational Linguistics: EMNLP 2023, 2023.

Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In Advances in Neural Information Processing Systems 37, 2024.

Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In Advances in Neural Information Processing Systems 36, 2023.

Sukannya Purkayastha, Sebastian Ruder, Jonas Pfeiffer, Iryna Gurevych, and Ivan Vulić. Romanization-based large-scale adaptation of multilingual language models. In Findings of the Association for Computational Linguistics: EMNLP 2023, 2023.

Minuri Rajapakse and Ruvan Weerasinghe. Script sensitivity: Benchmarking language models on Unicode, romanized and mixed-script Sinhala. arXiv preprint arXiv:2601.14958, 2026.

Alan Saji, Jaavid Aktar Husain, Thanmay Jayakumar, Raj Dabre, Anoop Kunchukuttan, and Ratish Puduppully. RomanLens: The role of latent romanization in multilinguality in LLMs. In Findings of the Association for Computational Linguistics: ACL 2025, 2025.

Rajvee Sheth, Himanshu Beniwal, and Mayank Singh. COMI-LINGUA: Expert annotated large-scale dataset for multitask NLP in Hindi-English code-mixing. arXiv preprint arXiv:2503.21670, 2025.

Harman Singh, Nitish Gupta, Shikhar Bharadwaj, Dinesh Tewari, and Partha Talukdar. IndicGenBench: A multilingual benchmark to evaluate generation capabilities of LLMs on indic languages. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024.

Guijin Son, Dongkeun Yoon, Juyoung Suk, Javier Aula-Blasco, Mano Aslan, Vu Trong Kim, Shayekh Bin Islam, Jaume Prats-Cristià, Lucía Tormo-Bañuelos, and Seungone Kim. MM-Eval: A multilingual meta-evaluation benchmark for LLM-as-a-Judge and reward models. arXiv preprint arXiv:2410.17578, 2024.

Priyansh Srivastava. The tokenizer tax: Quantifying and explaining the cross-lingual cost of subword tokenization for indian languages. arXiv preprint arXiv:2607.24276, 2026.

Sshubam Verma, Mohammed Safi Ur Rahman Khan, Vishwajeet Kumar, Rudra Murthy, and Jaydeep Sen. MILU: A multi-task indic language understanding benchmark. In Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, 2025.

Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Qi Liu, Tianyu Liu, and Zhifang Sui. Large language models are not fair evaluators. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024.

Ishaan Watts, Varun Gumma, Aditya Yadavalli, Vivek Seshadri, Manohar Swaminathan, and Sunayana Sitaram. PARIKSHA: A large-scale investigation of human-LLM evaluator agreement on multilingual and multi-cultural data. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024.

Orgest Xhelili, Yihong Liu, and Hinrich Schütze. Breaking the script barrier in multilingual pre-trained language models with transliteration-based post-training alignment. In Findings of the Association for Computational Linguistics: EMNLP 2024, 2024.

Yilun Yang and Yekun Chai. CodeMixBench: Evaluating code-mixing capabilities of LLMs across 18 languages. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025.

Jiayi Ye, Yanbo Wang, Yue Huang, Dongping Chen, Qihui Zhang, Nuno Moniz, Tian Gao, Werner Geyer, Chao Huang, Pin-Yu Chen, Nitesh V. Chawla, and Xiangliang Zhang. Justice or prejudice? Quantifying biases in LLM-as-a-Judge. In The Thirteenth International Conference on Learning Representations (ICLR), 2025.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track, 2023.

Ej Zhou, Lucas Resck, Zheng Hui, and Anna Korhonen. Lower-resource, higher scores: Language bias in LLM evaluators. arXiv preprint arXiv:2607.14480, 2026a.

Hongli Zhou, Hui Huang, Rui Zhang, Kehai Chen, Bing Xu, Conghui Zhu, Tiejun Zhao, and Muyun Yang. Toward robust LLM-based judges: Taxonomic bias evaluation and debiasing optimization. arXiv preprint arXiv:2603.08091, 2026b.

Irune Zubiaga, Aitor Soroa, and Rodrigo Agerri. Towards reliable multilingual LLMs-as-a-Judge: An empirical study. arXiv preprint arXiv:2605.28710, 2026.

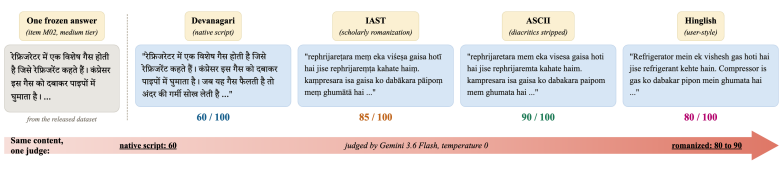

Figure 6: Qualitative example (item M02). Same content, four orthographies, one judge: scores of 60 (Devanagari), 85, 90, and 80 (romanized), verbatim from the released data.

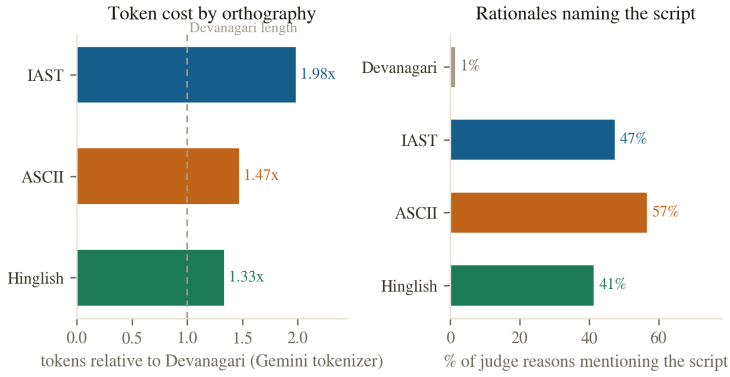

Figure 7: Left: token cost relative to Devanagari, Gemini tokenizer, $n = 150$. Right: share of judge rationales naming the script, Gemini 3.6 Flash, $n = 150$ per condition.

A Verbatim judging prompts

Main rubric (all core arms; {instruction} and {response} substituted per item):

> You are grading one response to one instruction for an instruction-following benchmark.
> Score the response from 0 to 100 considering: correctness (facts right), completeness (covers what the instruction asks), helpfulness, and clarity.
> Judge this item on its own merits.
> Instruction: {instruction}
> Response: {response}
> Reply with ONLY a JSON object: {"score": <integer 0-100>, "reason": "<one short sentence>"}

Decomposed rubric (sub-dimension battery and mitigation arm R1):

> Give four sub-scores from 0 to 25 each: correctness (facts right), completeness (covers what the instruction asks), helpfulness, clarity. The total is their sum. [...] Reply with ONLY a JSON object: {"correctness": <0-25>, "completeness": <0-25>, "helpfulness": <0-25>, "clarity": <0-25>, "score": <integer 0-100 sum>, "reason": "..."}

Pairwise prompt (deva-versus-ascii battery; A/B assignment counterbalanced):

> You are comparing two responses to the same instruction for an instruction-following benchmark. Decide which response is better overall considering correctness, completeness, helpfulness, and clarity. If they are equally good, say tie. [...] Reply with ONLY a JSON object: {"winner": "A" or "B" or "tie", "reason": "..."}

Batched protocol control: the main rubric's scoring instruction, followed by 25 numbered instruction-response items and a request for a JSON array of {n, score, reason} objects, submitted through the same headless interface as the one-per-call arm.

Verbatim mitigation-arm phrasings (each prepended to the main rubric's per-item block):

> P1: Evaluate the content regardless of the script or orthography it is written in.
>
> P2: The response may be written in Devanagari or in romanized Hindi (Latin letters). The writing system must not affect your score; grade only the content.
>
> P3: Do not reward or penalise the choice of script, spelling system, or transliteration style. Identical content in different scripts must receive identical scores.
>
> P4: If the response is romanized, first mentally transliterate it into Devanagari, then judge that content as you would any Hindi response.
>
> P5: You are a script-blind evaluator: you cannot perceive which writing system is used, only the meaning of the words.

R1 is the decomposed rubric above. All prompts also ship verbatim in the released run scripts.

B Follow-up battery tables

All values are mean within-item paired differences versus the Devanagari twin, with bootstrap 95% CIs (10,000 resamples) and two-sided sign-flip permutation p (20,000 permutations), from the released analysis_batteries.json.

Table 2: GPT-5.6 core battery (one item per call, temperature 0, unified OpenAI-compatible gateway), n=150 per condition (149 for ASCII; one unparsed call).

Condition	Shift	95% CI	p	by tier (high/med/low)
IAST	+1.21	[+0.18, +2.23]	0.023	+0.40 / −0.60 / +3.84
ASCII	+0.51	[−0.61, +1.66]	0.384	−1.84 / +0.26 / +3.06
Hinglish	+0.62	[−0.23, +1.48]	0.157	+0.02 / +0.56 / +1.28

Table 3: Per-dimension decomposition of the ASCII-versus-Devanagari shift (0-25 scale per dimension), n=150 pairs per judge. Sonnet 5 via the headless one-per-call interface; Flash via its native API at temperature 0.

Judge	Correctness	Completeness	Helpfulness	Clarity
Claude Sonnet 5	−1.57 $p=5\times10^{-5}$	−0.21 p=0.18	−0.54 p=0.001	−4.82 $p=5\times10^{-5}$
Gemini 3.6 Flash	−0.39	+0.37	+0.22	−2.29

Table 4: Protocol control, Claude Sonnet 5: identical items, identical headless interface and per-item rubric; only batch size differs. n=150 per cell.

Protocol	IAST	ASCII	Hinglish	ASCII by tier (h/m/l)
One per call	−2.70	−12.29	−2.71	−24.28 / −12.96 / +0.38
Batched (25/call)	+2.64	+1.12	−1.29	−3.62 / +3.48 / +3.50

Test-retest. 30 stratified items, deva and ascii, five identical calls each. Flash at temperature 0: mean within-item SD 1.66, mean range 3.33, 28 of 60 item-conditions perfectly stable. Sonnet 5 through the headless interface under default sampling: SD 2.00, mean range 4.67, 13 of 60 perfectly stable.

Anchored pointwise (Flash, deva original as in-context reference, n=150 pairs): ASCII minus deva −0.97 overall ([−4.9, +3.1], p=0.65); by tier −10.4 (high), −4.2 (medium), +11.7 (low). The deva cell's response equals its reference, so we read this battery directionally.

FDR correction. Benjamini-Hochberg over all 198 reported p-values: 108 remain significant at q=0.05, including every effect named in the abstract. The full per-test table ships in analysis_extra.json.

Logit-scale tier reanalysis (Flash). Transforming scores by $\text{logit}(s/100)$ (clipped to $[0.5, 99.5]$) before differencing removes bounded-scale compression. The medium tier retains the largest shift under all three romanizations (logit shifts +0.29 to +0.35, versus at most +0.22 for high and low tiers, and −0.48 for high-tier ASCII), so the discretion account survives the transform.

Qwen leading-token distribution. Top-20 logprobs at the first score token (leading digit for two-digit scores): expectation 5.70 (deva), 6.33 (IAST), 6.17 (ASCII), 5.73 (Hinglish); within-call SD 0.60 to 0.62 throughout, consistent with a mode move rather than a widening.

Table 5: Cross-script control: deliberately mismatched instruction/response cells versus the matched cells of the core arms. n=150 per cell, both $p = 5 \times 10^{-5}$ except where shown.

Mismatched cell vs matched baseline	Flash	Sonnet 5
Deva instruction + ASCII response, vs matched ASCII	−13.50 [−15.6, −11.5]	−16.08 [−19.6, −12.5]
IAST instruction + Deva response, vs matched Deva	+3.10 [+2.1, +4.2]	−3.71 [−5.5, −2.0]

Table 6: Mean within-item shift versus Devanagari by quality tier (high / medium / low, n=50 per tier), one-item-per-call arms: the numbers behind Table 1 and Figure 3.

Judge	Condition	High	Medium	Low
Gemini 3.6 Flash	IAST	+0.50	+7.80	+2.00
	ASCII	−1.30	+7.40	+0.40
	Hinglish	+0.20	+6.50	+2.70
Gemini 3.1 Pro	IAST	0.00	+2.20	+1.30
	ASCII	−0.70	+2.10	+1.90
	Hinglish	0.00	+2.70	+0.70
Qwen2.5-7B	IAST	−1.50	+6.60	+17.80
	ASCII	−1.90	+2.60	+17.40
	Hinglish	−0.20	−1.40	+1.80
Claude Sonnet 5	IAST	−6.74	−3.80	+2.44
	ASCII	−24.28	−12.96	+0.38
	Hinglish	−5.58	−3.90	+1.34
Claude Opus 5	IAST	−1.02	+0.12	+0.50
	ASCII	−6.18	−4.08	−0.02
	Hinglish	−1.82	−1.94	+0.22
Claude Fable 5	IAST	−0.28	+2.78	+1.80
	ASCII	−3.78	+1.46	+1.42
	Hinglish	−0.06	−0.32	+0.78
GPT-5.6	IAST	+0.40	−0.60	+3.84
	ASCII	−1.84	+0.26	+3.06
	Hinglish	+0.02	+0.56	+1.28

C Per-tier shifts for every judge

D Mitigation battery, full per-arm table

Table 7: Gemini 3.6 Flash, mean shift versus Devanagari per mitigation arm (all items n=150; medium tier n=50 in parentheses). Qwen per-arm numbers are in §4.4.

Arm	IAST	ASCII	Hinglish
None (baseline)	+3.43 (+7.80)	+2.17 (+7.40)	+3.13 (+6.50)
P1 plain	+2.73 (+7.20)	+2.03 (+6.10)	+3.07 (+7.20)
P2 explicit warning	+2.33 (+5.20)	+1.73 (+5.20)	+2.23 (+5.30)
P3 fairness	+3.00 (+6.90)	+3.07 (+8.00)	+2.93 (+6.60)
P4 transliterate-first	+2.10 (+4.40)	+1.93 (+5.20)	+1.83 (+4.40)
P5 script-blind persona	+3.27 (+7.70)	+2.97 (+7.70)	+3.80 (+8.90)
R1 decomposed rubric	+1.47 (+2.78)	−2.36 (−2.76)	+1.37 (+2.14)

E Review-response batteries

Table 8: Matched-gateway Sonnet 5 core arm: the full battery rerun through the same OpenAI-compatible gateway as GPT-5.6, temperature 0, one item per call, n=150 per condition. Compare Table 1's headless-CLI row.

Condition	Shift	95% CI	p	by tier (h/m/l)
IAST	+0.49	$[-0.60, +1.63]$	0.40	−2.64 / +2.30 / +1.80
ASCII	−3.64	$[-5.02, -2.26]$	5×10^{-5}	−9.24 / −2.56 / +0.88
Hinglish	+1.89	$[+0.92, +2.94]$	5×10^{-5}	−0.24 / +5.28 / +0.64

Table 9: Authorship-replication set: 51 items authored by Gemini 3.1 Pro (17 per tier), deva/IAST/ASCII, script-matched instructions, one item per call.

Judge	Condition	Shift	95% CI	p	by tier (h/m/l)
Sonnet 5 (CLI)	IAST	+2.92	$[-0.18, +5.98]$	0.072	−2.82 / +4.29 / +7.29
	ASCII	−4.27	$[-9.37, +0.98]$	0.118	**−16.71** (p=10^{-4}) / −3.35 / +7.24
Flash	IAST	+4.61	$[+2.65, +6.86]$	5×10^{-5}	+0.29 / +5.59 / +7.94
	ASCII	+2.75	$[+0.69, +5.10]$	0.020	0.00 / +2.35 / +5.88

Table 10: Pairwise deva-versus-ascii preference, all seven judges, 300 order-counterbalanced trials each.

Judge	Deva wins	ASCII wins	Ties	Sign-test p
Gemini 3.6 Flash	288	0	12	4.0×10^{-87}
Gemini 3.1 Pro	299	0	1	2.0×10^{-90}
Qwen2.5-7B	252	8	40	5.2×10^{-64}
Claude Sonnet 5	293	0	7	1.3×10^{-88}
Claude Opus 5	300	0	0	9.8×10^{-91}
Claude Fable 5	300	0	0	9.8×10^{-91}
GPT-5.6	283	0	17	1.3×10^{-85}

Table 11: Ranking demo: three real systems' Hindi answers to 50 instructions, scored by Sonnet 5 (non-contestant) in both scripts, n=50 per cell.

System	Devanagari mean	ASCII mean	Script shift
Gemini 3.6 Flash	62.24	56.18	−6.06
Gemini 3.1 Pro	57.06	51.88	−5.18
Qwen2.5-7B	12.78	15.62	+2.84